Projekt: Browary Bilba – Algorytm Successive Shortest Path z Dijkstrą i potencjałami

Autor: Łukasz Pniewski

Data wykonania testów: 01.06.2025

Środowisko testowe

Język programowania: C++

Platforma: Linux / Windows

Kompilator: g++

Biblioteki: “doctest.h”

Framework testowy: doctest v2

Cel testów

Celem testów jest potwierdzenie poprawności algorytmu *Successive Shortest Path* z Dijkstrą i potencjałami w kontekście wyznaczania minimalnego kosztu wcześniej wyznaczonego maksymalnego przepływu, poprzez algorytm Edmondsa-Karpa oraz sprawdzenie, czy rozważany przepływ zgadza się z tym wcześniej obliczonym, na końcu działania algorytmu. Sprawdzana jest poprawność wyników oraz aktualizacji potencjałów, które eliminują ryzyko ujemnych kosztów. Testy obejmują przypadki graniczne, takie jak wąskie gardła czy cofanie przepływu, aby upewnić się, że algorytm działa stabilnie i niezawodnie w różnych konfiguracjach podanego grafu.

Zakres testów

1) Dijkstra(graf, koszty_naprawy, rodzic, koszty_dojscia, potencjaly, source, target)

- poprawne stwierdzenie o znalezieniu ścieżki,

- poprawne stwierdzenie o braku ścieżki

- odpowiednie wypełnianie tablicy ***rodzic***, przetrzymującej znalezioną ścieżkę w postaci przypisania poprzedników każdemu wierzchołkowi

- odpowiednie wypełnianie tablicy ***koszty_dojscia***, przetrzymującej dodany koszt dojscia do danego wierzcholka w danej iteracji algorytmu

2) aktualizacjaPotencjalow(rozmiar, koszty_dojscia, potencjaly)

- poprawne aktualizowanie potencjałów, tak ważnych dla uniknięcia ujemnych kosztów w algorytmie Dijkstry

3) SSP(graf, koszty_naprawy, source, target)

- znalezienie oczekiwanego minimalnego kosztu przesyłu ładunku z maksymalnego przepływu, który znaleziono poprzez algorytm Edmondsa-Karpa

- potwierdzenie równości przepływu, na którym algorytm zakończył pracę z tym znalezionym za pomocą algorytmu Edmondsa-Karpa

Wykonane testy

1) Dijkstra z potencjałami

a) Minimalny przepływ, jedna ścieżka

Test sprawdza, czy algorytm znajduje pojedynczą poprawną ścieżkę powiększającą w najprostszym przypadku – kiedy istnieje tylko jedno możliwe przejście ze źródła do ujścia. Przy tym sprawdza aktualizowanie tablicy **koszty_dojscia**.

b) Brak polaczenia karczmy z target

Test weryfikuje, czy algorytm poprawnie zwraca brak ścieżki, gdy między źródłem a ujściem nie istnieje żadne połączenie.

c) Graf zawierający cykl + różne przepustowości

Sprawdza, czy algorytm poprawnie działa w obecności cykli oraz poprawnie wybiera najkrótszą ścieżkę pomimo zróżnicowanych przepustowości krawędzi.

d) Graf z krawędziami odwrotnymi, iteracja inna niż pierwsza

Ten test bada, czy algorytm poprawnie uwzględnia odwrotne krawędzie odwrotne w dalszych iteracjach, po tym jak pierwotna przepustowość zostanie wykorzystana. Bada również, czy tablica **koszty_dojscia** jest poprawnie wypełniana, jeśli **potencjaly** były już wcześniej aktualizowane.

e) Użycie krawędzi odwrotnej do znalezienia ścieżki

Celem testu jest wymuszenie sytuacji, w której najlepsza ścieżka wiedzie przez krawędź odwrotną – czyli cofnięcie wcześniej przesłanego przepływu w celu uzyskania lepszej konfiguracji.

f) Wszystkie możliwe połączenia w grafie, zerowe koszty

Ten test sprawdza stabilność i deterministyczność działania algorytmu w sytuacji, gdzie dowolna ścieżka jest równie dobra kosztowo – wszystkie krawędzie mają zerowy koszt.

2) Aktualizacja potencjałów

a) Aktualizowanie potencjałów

Test izoluje mechanizm aktualizacji potencjałów i weryfikuje, czy po iteracji Dijkstry wartości potencjałów są poprawnie zwiększane za pomocą tablicy **koszty_dojscia**, zapewniając stabilne przekształcanie kosztów w późniejszych iteracjach.

3) Successive Shortest Path

a) Przepływ 1, jedna ścieżka

Sprawdza podstawowy przypadek, w którym możliwy jest tylko jeden przepływ przez jedną dostępną ścieżkę.

b) Użycie krawędzi odwrotnej

Ten test wymusza cofnięcie wcześniej przesłanego przepływu przez krawędź odwrotną. Sprawdza poprawne zarządzanie rezydualnym grafem, potencjałami i kosztami.

c) Wszystkie możliwe połączenia w grafie, zerowe koszty

Sprawdza stabilność i deterministyczność działania algorytmu, gdy każdy wierzchołek jest możliwie połączony z każdym innym, a wszystkie krawędzie mają zerowy koszt.

d) Wąskie gardło

Testuje sytuację, w której jedna krawędź o niskiej przepustowości ogranicza całkowity możliwy przepływ.

e) Graf zawierający cykl

Sprawdza poprawność działania algorytmu w obecności cykli – czy nie dochodzi do zapętlenia przepływu i czy cykl nie zaburza kosztów ścieżek powiększających.

f) Brak przepływu

Weryfikuje, czy algorytm poprawnie wykrywa sytuację, w której nie da się przesłać żadnego przepływu z powodu braku ścieżek lub zerowych przepustowości.

g) Graf z wielkimi i małymi przepustowościami

Sprawdza, czy algorytm poprawnie obsługuje grafy z dużym rozrzutem w wartościach przepustowości i kosztów.

h) Wiele zbędnych wierzchołków w grafie

Testuje odporność algorytmu na obecność wielu wierzchołków, które nie są częścią żadnej ścieżki ze źródła do celu.

Wyniki testów

Testy przeprowadzone – 15

Testy zakończone sukcesem – 15

Testy zakończone niepowodzeniem – 0

Przeprowadzone asercje – 107

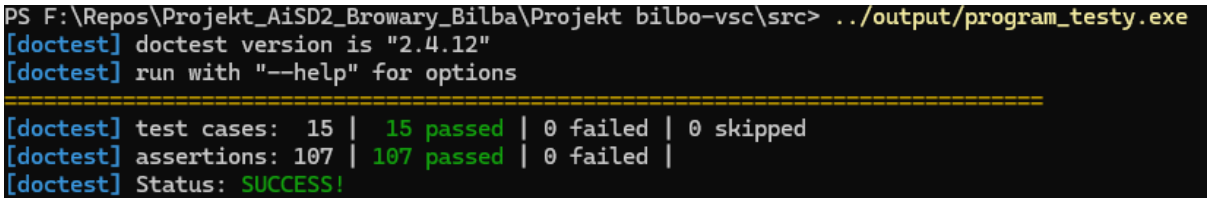
Asercje zakończone sukcesem – 107

Asercje zakończone niepowodzeniem – 0

Wykryte defekty: Brak

Pokrycie funkcjonalne: Wysokie (na podstawie analizy testowanych funkcjonalności)

Wynik z konsoli:



Podsumowanie

Wszystkie testy zostały wykonane w celu zweryfikowania poprawności oraz odporności algorytmu na różne układy danych sieci rezydualnych. Przetestowano zarówno podstawowe przypadki, jak i bardziej złożone scenariusze, takie jak obecność cykli, krawędzi rezydualnych, wąskich gardeł czy dużej liczby zbędnych wierzchołków. Każdy test potwierdził zgodność wyników z oczekiwaniami, a algorytm prawidłowo obliczał minimalny koszt przy maksymalnym przepływie w każdym z przypadków. Szczególnie istotne było prawidłowe zarządzanie potencjałami, które zapewniło bezpieczne działanie Dijkstry w obecności kosztów ujemnych. Pozwoliło to na poprawne znajdowanie ścieżek powiększających. Na podstawie uzyskanych rezultatów można stwierdzić, że implementacja jest stabilna, dokładna i spełnia założone wymagania funkcjonalne.